

CSC 667 - Milestone 3

Team z-f

Game: Uno

Team Members: Jonathan Mai, Ivan Wu, Megha Rai

Game Description

The game will be Uno. Players will be able to join a game and be able to draw cards and play cards from their hand.

[Socket.io](#) Events

Event: game:player:joined

Scope: all players in game room

Trigger: when any player successfully joins the room

Data:

```
{  
  Game_id: number.  
  Player_id: number,  
}
```

Event: game:player:left

Scope: all players in game room

Trigger: when any player successfully leaves the room

Data:

```
{  
  Game_id: number.  
  Player_id: number,  
}
```

```
}
```

Event: game:ended

Scope: all players in game room

Trigger: when a player wins the game

Data:

```
{  
Game_id: number.  
Player_id: number,  
}
```

Event: game:hand:update

Scope: the player whose hand is changed

Trigger: when a player draws a card

Data:

```
{  
Game_id: number.  
Player_id: number,  
Card_id: number,  
}
```

Event: game:turn:changed

Scope: all players in the room

Trigger:when a finishes all actions they can perform on their turn

Data:

```
{  
Game_id: number.  
Player_id: number,  
Updated_board: [...],
```

```
Next_player_id: number  
}
```

Event: game:state:update

Scope: all players in game room

Trigger: when the state of the game is changed

Data:

```
{  
Game_id: number.  
Player_id: number,  
Card_id: number,  
Updated_board: [...],  
Next_player_id: number  
}
```

Event: game:card:played

Scope: all players in game room

Trigger: When any player successfully plays a card

Data:

```
{  
  game_id: number,  
  player_id: number,  
  card_id: number,  
  updated_board: [...],  
  next_player_id: number  
}
```

Event: game:card:drawn

Scope: the player who drew the card

Trigger: When any player successfully draws a card

Data:

```
{
  game_id: number,
  player_id: number,
  card_id: number,
  next_player_id: number
}
```

API endpoints

Game Management Endpoints

a) Create Game

1. Endpoint Name: Create Game
2. HTTP Method & Route POST /api/games/
3. Purpose: Create a new UNO game lobby for people to join.
4. Authorization:
 - User must be authenticated
5. Request Body: { "privacy": { "public", "private", "friends" } }
6. Validation Checks:
 - User has a user_id
 - maxplayers is present and between 2 and 4
 - Privacy is either "public", "private", "friends"
7. State Updates:
 - Insert into game table
 - Records updated: room_id, maxplayers, status, privacy, created_at
 - Insert into game_user
 - Records updated: game_id, user_id, hand_count, seat_at
8. Success Response:
 - 201 created, with the game table returned
 - { "game_id", "room_id", "maxplayers", "privacy", "status" }
9. Error Cases
 - 401 Unauthorized
 - 400 Bad Request
 - 409 Conflict
10. Socket.io Events

- None
-

b) Join Game

1. Endpoint Name: Join Game
 2. HTTP Method & Route: POST /api/games/:game_id/join
 3. Purpose: Add a user as a player to a lobby
 4. Authorization:
 - User must be authenticated
 - The game must be visible according to the room privacy
 5. Request Body: None
 6. Validation Checks:
 - A game with game_id exists
 - game.status === open
 - The number of players in game does not exceed maxplayers
 - The joining user is not already a game_user
 7. State Updates:
 - Insert a row into the game_user table
 - game_id
 - user_id
 - hand_count = 0
 - seat_at = next lowest int
 8. Success Response:
 - 202 accepted
 - {game_user_id, hand_count, seat_at}
 9. Error Cases
 - 400 Bad Request - trying to join after the game has started
 - 401 Unauthorized - Not logged in
 - 403 Forbidden - privacy rules not satisfied
 - 404 Not Found - Game doesn't exist
 - 409 Conflict - game full or already in match
 10. Socket.io Events
 - game:player:joined → All players in the room
-

c) Start Game

1. Endpoint Name: Start Game
2. HTTP Method & Route: POST /api/games/:game_id/start
3. Purpose: turns the lobby into an active match/game
4. Authorization:
 - User must be authenticated
 - User must be a player in this game

5. Request Body:
 - {"starting_hand_size": 7}
6. Validation Checks:
 - Game with game_id exists
 - Game status === 'open'.
 - The owner of the lobby is in the game
 - The number of players is between 2 and 4
7. State Updates:
 - Update game table
 - status = 'inMatch'
 - direction = 'clockwise'
 - start_time = Current time
 - turn_number = randomly chosen number
8. Success Response:
 - 202 Accepted
 - {"game_id": 123, "status": "inMatch"}
9. Error Cases
 - 401 Unauthorized - Not logged in
 - 403 Forbidden - not in game or not the host
 - 404 Not Found - game not found
 - 409 Conflict - game is started, closed, or finished
10. Socket.io Events
 - game:state:update → All players in game
 - game:hand:update → Private to each player

d) Get Game State

1. Endpoint Name: Get Game State
2. HTTP Method & Route: GET /api/games/:game_id
3. Purpose: This is to get the current state of the game and get the player's state
4. Authorization:
 - User must be authenticated
 - Only allow players who meet the privacy requirements
5. Request Body: None
6. Validation Checks:
 - Game with game_id exists
 - Game_user.game_id === game.id
7. State Updates:
 - Nothing, read only
8. Success Response:
 - 200 OK

- The return body should have queried
 - The specified game row
 - The players who are in the game
 - The hands of each person in the game
 - 9. Error Cases
 - 401 Unauthorized - Not logged in
 - 403 Forbidden - not allowed to be in the game
 - 404 Not Found - Game doesn't exist
 - 10. Socket.io Events
 - None, read only
-

e) Leave Game

1. Endpoint Name: Leave Game
 2. HTTP Method & Route: POST /api/games/:game_id/leave
 3. Purpose: letting a player leave a lobby
 4. Authorization:
 - User must be authenticated
 5. Request Body: None
 6. Validation Checks:
 - A game with game_id exist
 - game.status === open
 - User is a game_user
 7. State Updates:
 - Remove the game_user entry with User.id === game_user.user_id
 8. Success Response:
 - 202 accepted
 9. Error Cases
 - 401 Unauthorized - Not logged in
 - 404 Not Found - Game doesn't exist
 - 400 Bad Request - User is not in the game
 10. Socket.io Events
 - game:player:left → All players in the room
-

Game Action Endpoints

a) Draw Card

1. Endpoint Name: Draw Card
2. HTTP Method & Route: POST /api/games/:game_id/draw
3. Purpose: Allow the player to draw one or more cards from a draw pile

4. Authorization:
 - User must be authenticated
 - User must be a player in this game
 - Has to be their turn (exception when penalized from +2 or +4)
5. Request Body: { "count": 1 } // count may change depending on penalty
6. Validation Checks:
 - Game exist, and is 'inMatch'
 - Game_user is a member of game
 - It is the game_user's turn (unless penalized)
 - There are cards in the draw pile
7. State Updates:
 - Selects the top N number of cards from game_cards, where card_pile = 'Draw'
 - Each selected card will be set to card_pile = 'Hand'
 - Owned_by will also be set to the current game_user
 - Game_user's hand will increment N times
8. Success Response:
 - 202 Accepted
9. Error Cases
 - 401 Unauthorized - Not logged in
 - 403 Forbidden - Not their turn
 - 404 Not Found - Game doesn't exist
10. Socket.io Events
 - game:card:drawn → private to the drawing player
 - game:hand:update → private to the drawing player
 - game:state:update → All players in game, updates draw pile, and hand count

b) Play card

1. Endpoint Name: Play card
2. HTTP Method & Route: POST /api/games/:game_id/cards/:card_id/play
3. Purpose: Player plays a card from their hand onto the game board
4. Authorization:
 - User must be authenticated
 - User must be a player in this game
 - Must be the current user's turn
 - Game must be in "inMatch" state
5. Request Body:
 - Client sends in the color they choose if the played card is a wild card
 - { "chosen_color": "Blue" }
6. Validation Checks:
 - Game with game_id exists

- User is in the game_user table for this game
 - The current game user matches the game_user.id
 - game.state === "inMatch"
 - The card exists in a player's hands ie. card_pile === 'Hand'
 - The card is owned_by === game_user.id
 - If not first turn, then
 - The card's color value matches the top of the discard pile or
 - The card's number value matches the top of the discard pile or
 - The card is a Wild/WildDrawFour
 - The card's chosen color is a valid color
7. State Updates:
- Move game_cards.id from hand to discard
 - game_cards.id = game_card_id
 - card_pile = 'Discard'
 - owned_by = NULL
 - Update game_user.hand_count -= 1
 - Update the top discard card with the played card
8. Success Response:
- 202 Accepted
9. Error Cases
- 401 Unauthorized - Not logged in
 - 403 Forbidden - Not your turn or not in game
 - 404 Not Found - Game or card doesn't exist
 - 400 Bad Request - Invalid move, illegal play
 - 409 Conflict - Game state changed since request started
10. Socket.io Events
- game:state:update → All players in game
 - game:hand:update → Player who played card only
 - game:uno:called → All players in the game room

c) Call Uno

1. Endpoint Name: Call Uno
2. HTTP Method & Route: POST /api/games/:game_id/uno
3. Purpose: Player signals that they have one card left
4. Authorization:
 - User must be authenticated
 - User must be a player in this game
 - Game must be in "inMatch" state
5. Request Body: None
6. Validation Checks:

- Game with game_id exists
 - User is in the game_user table for this game
 - game_user has exactly 1 card left
7. State Updates:
- None
8. Success Response:
- 202 Accepted
9. Error Cases
- 401 Unauthorized - Not logged in
 - 403 Forbidden - Not in game
 - 404 Not Found - Game doesn't exist
 - 400 Bad Request - Invalid move, illegal play
10. Socket.io Events